

Privacy-Preserving Inclusion Lists

Zhengwei Tong¹, Saba Eskandarian², and Kartik Nayak¹

¹ Duke University, Durham, NC, USA

`zhengwei.tong@duke.edu`, `kartik@cs.duke.edu`

² University of North Carolina at Chapel Hill, NC, USA

`saba@cs.unc.edu`

Abstract. Blockchains aim to provide open access and censorship resistance, but centralization of block production in blockchains like Ethereum undermines these goals. Inclusion list protocols have been proposed to mitigate censorship by requiring block proposers to include transactions proposed by a committee of inclusion list (IL) proposers. However, protecting the privacy of these proposers is essential to prevent retaliation and to ensure robust censorship resistance.

We propose a lightweight, privacy-preserving inclusion list protocol that allows committees to collectively construct transaction lists while hiding individual contributions and ensuring plausible deniability. Our approach builds on multiparty computation (MPC) techniques to achieve strong privacy without relying on heavyweight cryptography or anonymous channels. We implement two variants of these protocols that support a third of the parties behaving adversarially: an optimistic version that supports malicious security with abort, and a robust MPC that tolerates arbitrary faults. Under a default parameterization for inclusion lists being considered today, we observe a latency of ~ 5.1 s and ~ 432.7 s in the two variants.

1 Introduction

Blockchains provide the ideal functionality of a general-purpose computer that is available and accessible to anyone, has no owner or operator, and supports property rights for user-owned data. However, implementations differ from such an idealized world, and there exist forces of centralization due to which these goals are not achieved. For instance, many blockchain protocols use block proposers or leaders that are responsible for selecting transactions added to the blockchain; thus, at any time, they temporarily control the availability and accessibility of the blockchain. In fact, in blockchains such as Ethereum, block production is outsourced to entities called builders, and the top-3 builders are responsible for producing more than 90% of the blocks [28,27,29]. Ironically, this is similar to the centralized credit card networks where 95% of the purchase volume is controlled by the top three networks (Visa, Mastercard, and American Express) in the United States [1].

A key concern related to the accessibility of the blockchain is censorship: a valid transaction proposed by any party must be included in the chain soon

enough, and censorship is the act of refusing valid transactions in the chain. There are two key reasons for censorship: self-censorship due to regulation and financial incentives. In recent years, we have observed self-censorship in the form of OFAC sanctions [23]. Moreover, applications related to decentralized finance can create financial incentives for censoring [25,26,13]. However, for a system intended to be accessible to anyone and everyone, we need to ensure that transactions cannot be censored by potential centralizing entities. At the same time, parties that uphold the ideal functionality and provide censorship resistance should not be at risk of exposure or harm for doing so.

In blockchains such as Ethereum, censorship is proposed to be tackled through a separate sidecar protocol called an *inclusion list protocol* that produces an agreed-upon list of transactions, and the blockchain proposer is required to force-include the output of the inclusion list protocol [22]. A key approach to prevent censorship in this protocol itself is to use a committee of *multiple proposers* (also called IL proposers) and output transactions that are proposed by a small threshold of the proposers. Having multiple parties ensures greater decentralization, and thus, it is harder for transactions to be left out if a threshold of the parties are honest. Consequently, the job of the honest parties in the committee is to propose transactions in the mempool that are being censored. To boost the adoption of inclusion lists, in an ideal world, we need to protect the identity of these proposers, akin to the principle of protecting confidentiality in voting systems; this way, we can safeguard the honest parties against any form of punishment for supporting censorship resistance, and parties can uphold the ideal properties of the protocol without the risk of exposure or retaliation [2]. The goal of this work is to design such a multi-proposer privacy-preserving inclusion list protocol that incorporates transactions proposed by a threshold of parties while also protecting the privacy of parties proposing these transactions.

Key challenges and desiderata. There are several key challenges to designing such a privacy-preserving inclusion list. A core challenge involves achieving the desired privacy property that de-links a transaction input by a party from the output of the committee, even if some parties deviate from the protocol and behave arbitrarily. Thus, a proposer obtains the necessary anonymity guarantees and it can plausibly deny having included any transaction. There are several broad cryptographic approaches to solve this problem, such as the use of DC nets [8,16,9,10], multiparty computation [15,4,11,19,18], and the use of linkable ring signatures [6,17]. However, the goal is to ensure that the cryptographic tools used achieve the desired privacy properties while reducing the amount of computation and communication needed. From a performance standpoint, the lower the latency of the inclusion list protocol, the sooner the censored transactions can be included in the main chain. Thus, if it takes time equivalent to k blockchain “slots” to produce one inclusion list, then our goal is to reduce this k ; ideally $k = 1$ and we are able to include a new inclusion list in every time slot. A final consideration is anonymity at the network level, i.e., we want to achieve these guarantees even against an adversary that is able to observe all network traffic.

Our solution. We present a protocol that allows an inclusion list committee to produce an inclusion list collectively without revealing any individual member’s contributions and while giving each member plausible deniability for introducing each transaction to the final inclusion list. The protocol is agnostic with regard to committee selection processes, assuming only that committee members can communicate with each other. At a high level, our protocol employs ideas from the literature on multiparty computation (MPC) [15,4,11,19,18] and private computation of heavy hitters [7] to allow committee members to develop a list of transactions that many members want to place on the inclusion list. Prior proposals use more heavyweight cryptographic tools and assume all committee members use an anonymous broadcast channel to communicate [2,3]. By comparison, our protocol uses only lightweight cryptography and requires no additional privacy properties from the communication infrastructure. We present a detailed technical overview of our protocol in Section 3.

Implementation and evaluation. We implement two versions of our protocol. Both provide all necessary privacy properties, but they differ in output delivery guarantees. The first is an optimistic version that uses the Mascot protocol in MP-SPDZ [19,18] and completes execution only when all of the committee members cooperate. When evaluated under parameters chosen consistent with EIP-7805 [22] and a 16-member IL committee, the protocol runs within 5.1 ± 0.1 s, which is faster than the rate at which blocks are currently produced. The second version is robust under a Byzantine threshold of $t < n/3$ committee members and runs in 432.7s. We present a detailed analysis in Section 4.

1.1 Closely Related Work

The goal in this work is inspired by the need for a list of privacy-preserving inclusions for the Ethereum blockchain [2,3]. There are two high-level proposals in the Ethereum community. The first is an “anonymous inclusion list” design that combines a committee-based design (akin to this work) with linkable ring signatures [17,6] to achieve anonymity [2]. The second design, called zkFOCIL, relies on linkable ring signatures based on general-purpose SNARKs [3]. Both of these proposals are yet to be fully specified, analyzed, and implemented. Moreover, unlike our protocol, neither of these proposals obtain network-level anonymity guarantees. We discuss these and other related approaches to solving this problem in Appendix C.

2 Problem Setting

2.1 Problem Statement

Our core problem involves a set of n parties $P_1, \dots, P_n$. We assume that up to t of the n parties are Byzantine, and thus can behave arbitrarily; the remaining parties are honest. Each of the n parties can communicate with each other using an authenticated channel.

There is a global set of values representing transaction hashes, where each value is a κ -bit value chosen uniformly at random. Each party P_i has an input list L_i containing a subset of these values. Our goal is for the parties to privately compute an intersection of all parties' input lists, and output a list L consisting of values that appear in a threshold number of inputs. Intuitively, the privacy goal is to ensure that an adversary cannot determine whether a specific party input a specific value based on the protocol transcript and the final outputs. More precisely, we want the following properties to hold for parameters y and τ :

- **Correctness.** If a value v is included in the inputs of at least y honest parties, then it must appear in the output despite arbitrary behavior by the t Byzantine parties.
- **Optimistic correctness.** If a value v is included in the inputs of at least y honest parties, then it must appear in the output if all the parties follow the protocol specification.
- **Soundness.** No value v will be included in the output L unless it is included in L_i for at least one honest party.
- **Anonymity.** Parties to the protocol learn at most the count of how many parties have included a value v in their list L_i , with no information leaked about which parties contributed to the count.
- **Plausible deniability.** For each value v , no coalition of $\leq t$ parties can determine with certainty whether a given party P_i included v in their input based on the protocol transcript and the outputs.

More precisely, for any output list L , value v , index i , and protocol transcript T , it is possible to present a list of honest parties' inputs $L_1, \dots, L_y$ where $v \notin L_i$ that is consistent with T , and that results in output L with constant probability.

At first glance, our anonymity and plausible deniability requirements may seem very similar, but they are subtly different. Anonymity states that committee members can in no way be tied to their particular votes, which is a strong requirement, but in the case where every single committee member includes a particular transaction hash v , this does not suffice for deniability. Deniability covers this edge case, but it has the weaker requirement that it is *possible* that a given committee member did not include a transaction v , not that it is just as likely as any other situation.

2.2 Privacy-Preserving Inclusion List for Blockchains

We now describe the larger context in which solving the above problem is relevant (cf. Figure 1). As described in the Introduction, in blockchains such as Ethereum, there are designated proposers for each *slot* of the protocol. Each slot can hold a block, and as of today, more than 90% of these blocks are built by two external entities called builders [28,27,29]. Such centralization makes it easy for transactions to be censored, either due to economic reasons or due to coercion [23,25,26,13]. The notion of an inclusion list is meant to address this

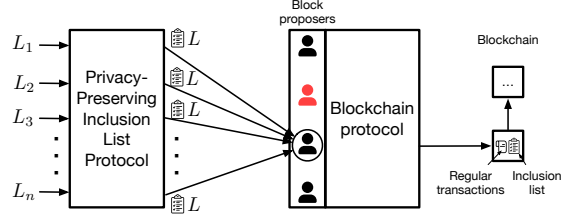


Fig. 1: Privacy-Preserving Inclusion List

problem by relying on a decentralized set of blockchain validators to agree on a set of censored transactions during a slot k , and the protocol requires the block proposer at slot $k + 1$ to include the agreed-upon set in the slot.

We rely on the existence of a committee of validators whose members hold responsibility for producing the inclusion list for each slot. The membership of the committee must be public, and there must be consensus around who is on the committee. While the mechanism for constructing the committee is outside the scope of this work, our proposed solution is compatible with any committee that satisfies these requirements. However, for decentralization purposes, it will be beneficial if the committee is chosen at random afresh for each slot.

The committee members form the n parties we referred to in the problem statement. We assume each of these parties must have access to the mempool over time in order to determine which transactions should go on an inclusion list. This is the reason why we assume each of the parties' inputs comes from a global set of values. We note that no individual party may know the exact global set. Moreover, while there will naturally be a large degree of overlap between committee members' views of the mempool, we do not require them to have identical views.

Protocol inputs and outputs. The committee members provide as input *transaction hashes*, and each member outputs an agreed-upon list of such hashes. Committee members sign the list of hashes and share them with proposers on the network. A list signed by more than t committee members (and thus, at least one honest committee member) is considered a valid inclusion list for the slot. In order to produce a valid block, proposers must show that for each transaction hash h on the inclusion list, they have included a transaction tx such that $H(\text{tx}) = h$ for the hash function H used by our scheme. Note that signing the outputs of the protocol is not connected to each committee member's individual inputs, so signatures on the output of the protocol do not reveal which, if any, of the included transactions were added by any individual committee member.

The hash function H can be any collision-resistant hash function of appropriate output length. Since the performance of our core protocol will depend on the output size of the hash function used, we can optimize performance by setting the output length to be shorter than is typically used for CRHFs, e.g., 64 bits, without affecting the privacy properties of the scheme. This introduces the possibility of occasional collisions where two different transactions hash to an entry on the inclusion list. We consider a block with either transaction to be valid, with the

understanding that if the “wrong” transaction is included, the protocol execution on the next slot will still include that hash on the inclusion list, and the “correct” transaction can be included the next time around. In Section 4, we explore parameter tradeoffs like this one and introduce optimizations that replace a single hash with multiple shorter hashes in order to improve parallelism in protocol executions.

Properties in the larger context. We make a few observations relating to the properties described in the problem statement and the larger context. First, we expect the proposer to include a transaction tx based on only the transaction hash $H(\text{tx})$ in the output. To guarantee this, we require that a transaction in the inclusion list is input by at least one honest party (soundness property), and thus, the corresponding transaction is available on the mempool. Looking ahead, our protocol will test that any transaction output has been included by at least $\tau \geq t + 1$ parties. We note that, if data availability was ensured through other means or it was not needed, we could have avoided the constraint on τ . Second, our correctness property guarantees that a transaction input by at least y honest parties must appear in the output. Given censorship is a motivation, our goal will be to minimize the value of y . Finally, our requirements of anonymity and plausible deniability ensures that parties can input censored transactions while remaining unlinkable to the transactions that they input individually (anonymity) and each being able to deny that they input any particular transaction (plausible deniability).

2.3 A Two-Track Approach

Our core technical contribution is a multiparty protocol for voting on and producing a list of transaction hashes that constitute the contents of an inclusion list. The protocol, described in Section 3, consists of a recursive voting/narrowing procedure over the space of all possible hashes that identifies which hashes are included in the individual inclusion lists of τ or more committee members, without revealing which members have which transactions on their lists.

A two-track solution. A large part of our core protocol involves running a series of small MPC subprotocols. The underlying MPC protocols used must provide malicious security and robustness due to the possibility of malicious parties on the committee tampering with the protocol or launching a denial of service attack by going offline. Unfortunately, known robust MPC protocols are more expensive than their counterparts that only provide malicious security with abort. The latter provides the guarantee that if an honest party gets an output, then it can check that it has indeed received the correct output, but it leaves room for malicious parties to interrupt the protocol by simply aborting [19,11].

We observe that since malicious security with abort suffices for our soundness and privacy requirements, we can adopt a two-track approach. We run two versions of the protocol in parallel. The first version is a faster MPC protocol that only provides security with abort, i.e., satisfies optimistic correctness, and outputs a value within a deterministic time bound under a synchronous network.

The other version provides correctness even in the face of arbitrary behavior by the t malicious parties. If the first version runs successfully, the output of that version will be considered for the slot; otherwise, we consider the output of the robust version, which will become available for a future slot.

Section 3 describes our privacy-preserving inclusion list protocol, and we provide additional details and optimizations in Appendices A and B, respectively. We briefly compare our approach to other potential approaches in Appendix C.

3 Protocol Description

Prior approaches to anonymous inclusion lists rely on ring signatures and assume an underlying anonymous broadcast protocol, separating out application and network level privacy concerns [2,3]. We observe that by collapsing these two levels of the protocol stack, we can construct solutions that simultaneously use more lightweight cryptographic primitives and provide tailor-made privacy guarantees, reducing the demands placed on the underlying communication protocol. Our protocol will only require standard encrypted/authenticated channels between parties.

We adapt ideas from multiparty computation [15,4,11,19,18] and anonymous communication [8] to produce lightweight privacy-preserving inclusion lists with a strong k -anonymity property: when a transaction is included in an inclusion list, nobody knows which member(s) of the inclusion list committee contributed the transaction. Moreover, we wish to satisfy a deniability property: any committee member should be able to plausibly argue that they did not add a given transaction to the inclusion list. Our scheme leaks only the (approximate) number of members who included each transaction on the final inclusion list. This is because the protocol allows the committee, subject to Byzantine inputs, to enforce arbitrary thresholds on how many members need to include a transaction for it to be on the list.

We discuss our solution below, first assuming a fully semihonest setting where all parties follow the protocol but corrupt parties wish to learn information based on the protocol transcript. Next, we will sketch how to lift the security of our protocol to tolerate Byzantine behavior.

Background: Multiparty computation. Multiparty computation (MPC) protocols let parties with private data jointly compute a function without revealing their inputs. Conceptually, this is like a trusted third party collecting secrets and returning the result, but in practice, the parties run a protocol that simulates this role. Inputs are secret-shared—split into pieces $[x]_1, \dots, [x]_n$ such that $\sum_i [x]_i = x$ —and operations on these shares allow secure addition, multiplication, and thus any arithmetic circuit. At the end, parties combine their shares to reveal the output. We express our protocol by describing the function $\mathcal{F}_{\text{MPC}}$ that the trusted party would compute based on the parties’ inputs; in our implementation, this function is realized by replacing it with an MPC protocol.

3.1 A Semihonest Secure Approach

We think of inclusion list construction not as a problem where a number of parties append transactions to a list, but rather as a problem of selecting which transactions from the list of all possible transactions should be included. Intuitively, this is the difference between a sushi restaurant waiter asking for each person's order versus a table filling out a sheet with their menu selections. In the former scenario, each order is directly tied to the customer who asked for it, whereas in the latter, the group as a whole is connected to all parts of the order. Loosely speaking, our protocol will allow a committee to produce the completed order sheet without leaking any one member's contribution.

Desired functionality. More formally, we logically map all possible transactions, whether or not the transaction really exists, into the exponentially large output space of a collision resistant hash function H . Each party's input to our protocol will be an exponentially large vector with 1s in indices corresponding to hashes of transactions. When an inclusion list committee member holds a list of transaction hashes $L' = (\text{tx}_1, \dots, \text{tx}_m)$ that it wants in the list, it logically transforms this list into a bit vector L of length 2^κ where an entry i holds 1 if there exists $\text{tx}_j \in L', j \in [m]$, such that $H(\text{tx}_j) = i$. All other entries are set to 0. Importantly, this exponentially long vector is a logical construct and never concretely realized.

We will use a *private sum* protocol for n committee members to go from $L_1, \dots, L_n$ to a single list $L \leftarrow \sum_{i=1}^n L_i$. Entries of L that are greater than some threshold τ will be added to the final inclusion list. Importantly, the protocol will allow the committee members to efficiently recover only the non-zero entries of the sum vector L , thereby doing work at most linear in the size of the inclusion list and logarithmic in the output space of the hash function to recover the list, rather than the exponential work of a naïve approach.

Construction details. Our high-level approach is inspired by work on private computation of *heavy hitters* [7] in data held secret shared across a number of parties. Our protocol will allow committee members to recursively explore levels of a binary tree with 2^κ leaves, where each internal node consists of an indicator bit revealing whether it is on the path to a non-zero leaf, and the non-zero leaves make up the elements of the list L . Our exploration algorithm will follow only the paths that lead to non-zero entries in L , without revealing any individual member's contribution.

At each level of the tree, committee members compute the sum of their values for each tree node using a simple MPC protocol that only consists of additions. The sums are revealed without anyone learning which members contributed a zero or one for each node. Once the sums are public, the members assemble a list of those nodes whose counts exceed the committee's chosen threshold τ for inclusion in the final list (which can be greater than or equal to one). These are the only nodes that lie on paths which could possibly have a non-zero value in their leaves. The process is repeated for the children of each node whose count is above the threshold. At the κ^{th} level of recursion, the remaining nodes are in fact


```

1:  $L_i$ : list of input transaction hashes for party  $P_i$ 
2:  $AP_l$ : Set of active prefixes at level  $l$ 
3:  $b$ : branching factor, a power of 2
4:  $\kappa$ : length of input transaction hashes
5:  $n$ : number of parties
6:  $\tau$ : threshold
7:  $x$ : integer parameter

8:  $\text{coin}_i \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$ 
9:  $\text{out}_{\text{coin}} := \mathcal{F}_{\text{sum}}(\text{coin}_1, \dots, \text{coin}_n)$ 
10: Let  $j$  denote the smallest index in  $\text{out}_{\text{coin}}$  s.t.  $n - x \leq \text{out}_{\text{coin}}[j] \leq n - (t + 1)$ 
11: if  $\text{coin}_i[j] = 0$  then ▷ zero-input party
12:    $L_i = \{\}$  ▷ Sets input to be empty

13:  $AP_0 := \{0, \dots, b - 1\}$ 
14: for level  $l := 0, \dots, \log_b(2^\kappa) - 1$  do
15:    $V_{i,l} := \{\}$  ▷  $P_i$ 's vote map for level  $l$ 
16:   for prefix  $p \in AP_l$  do
17:     if  $\exists tx \in L_i$  with prefix  $p$  then
18:        $V_{i,l} := V_{i,l} \cup (p, 1)$ 
19:     else  $V_{i,l} := V_{i,l} \cup (p, 0)$ 
20:    $V_{G,l} := \mathcal{F}_{\text{MPC}}(V_{0,l}, \dots, V_{n-1,l})$ 
21:    $AP_{l+1} := \{\}$ 
22:   for entry  $\in V_{G,l}$  with key  $p$  do
23:     if  $V_{G,l}[p] \geq \tau$  then
24:        $AP_{l+1} := AP_{l+1} \cup \{(p||0), \dots, (p||b - 1)\}$ 
25: Output  $AP_{\log_b(2^\kappa) - 1}$ 

```

Fig. 2: Computing the inclusion list.

<pre> 1: for $i := \{0, \dots, n - 1\}$ do 2: for each entry with key p in $V_{i,l}$ do 3: if $V_{i,l}[p] \cdot (1 - V_{i,l}[p]) \neq 0$ then 4: Abort/ignore input 5: for each entry with key p in $V_{-,l}$ do 6: $V_{G,l}[p] := \sum_i V_{i,l}[p]$ 7: Return $V_{G,l}$ </pre>	<pre> 1: for $i := \{0, \dots, n - 1\}$ do 2: for $j := \{0, \dots, n - 1\}$ do 3: if $\text{coin}_i[j] \cdot (1 - \text{coin}_i[j]) \neq 0$ then 4: Abort/ignore input 5: ▷ Summing each entry across parties 5: Return $\sum_i^n \text{coin}_i$ </pre>
--	--

Fig. 3: $\mathcal{F}_{\text{MPC}}$ (left) and $\mathcal{F}_{\text{sum}}$ (right): $\mathcal{F}_{\text{MPC}}$ is the level- l MPC algorithm. $\mathcal{F}_{\text{sum}}$ is the coin-tossing algorithm. Line 4 results in aborting the process when running an MPC that is secure with abort, whereas a robust MPC simply discards the inputs that fail the check.

the non-zero entries of the list L , computed without revealing who contributed to each entry.

Protocol description. The resulting protocol is described in Figure 2; we present a more general protocol with branching factor b . For now, we ignore

lines 8-12. The protocol runs for $\log_b 2^\kappa$ iterations (line 14). In the first iteration, the protocol considers b number of prefixes; thus representing a $\log_2 b$ prefix length of transaction hashes. In each iteration, we increase the prefix length by $\log_2 b$. A party P_i maintains a list of active prefixes AP_l to be considered for level l (line 13). Based on the active prefix AP_l , P_i creates a local list $V_{i,l}$ where each element of a list is a key-value pair with a prefix $p \in AP_l$ being the key and a boolean value indicating whether it has an input transaction $tx \in L_i$ with prefix p (lines 16-19). Thus, a value 1 indicates that the party is interested in exploring the path with prefix p based on its input L_i , and it will be explored if sufficiently many other parties also input 1 for the same prefix. To determine this, the parties engage in a multiparty computation protocol $\mathcal{F}_{\text{MPC}}$ (line 20) where party P_i inputs $V_{i,l}$ (described in Figure 3). A secure multiparty computation protocol ensures that the parties only learn the output of the function computed on the parties' inputs, and nothing else; thus, a party's interest in exploring a path based on its input is kept private. In a semi-honest version of this MPC protocol, $\mathcal{F}_{\text{MPC}}$ only adds input $V_{i,l}[p]$ for each prefix p to produce output $V_{G,l}$ (Figure 3 lines 5-6). Party P_i then processes the output $V_{G,l}$ to determine the set of prefixes p that appears $\geq \tau$ times; if a prefix p matches that criteria, we explore prefixes $(p||0), \dots, (p||b-1)$ in the next iteration (line 22). At the end of the $\log_b 2^\kappa$ -th iteration, $AP_{\log_b 2^\kappa - 1}$ stores the desired set of outputs.

Adding deniability. While this approach hides who included which transaction, it does not provide plausible deniability if the count for a transaction is equal to the number of committee members because in this case, every committee member must have included the transaction. We add plausible deniability by adding in a lightweight, randomized mechanism whereby certain parties, chosen privately and at random, always contribute zeros to the counts. Since any party can claim to be among the ones who only contributed zeros, plausible deniability is retained.

Our idea is for each party to privately flip a 0/1 coin. If the coin comes up 0, then this party is a *zero-input party* for the protocol and will always give zeros as its input for every sum, regardless of its actual inputs. After tossing their coins, the parties reused our private sum functionality to ascertain the number of zero-input parties. They retoss the coins if there are too many zero-input parties to actually get transactions on the list, or if there are too few zero-input parties to provide meaningful plausible deniability. To avoid extra communication rounds, we have the parties toss and sum many coins in parallel and use the first successful result. We present a more detailed description on this in Appendix A.2.

Parameter tuning for security and performance. The inclusion threshold τ should be set based on expectations of the number of Byzantine/malicious parties against which the committee expects to protect itself. For example, if there is an expectation of t Byzantine parties, the threshold can be set to $\tau = t + 1$ so that the Byzantine parties cannot corrupt or disrupt the output of the protocol, e.g., by not providing data corresponding to the transaction.

At any level of recursion, the number of subprotocols in progress is upper-bounded by the total number of items that need to be put in the final inclusion list, within a factor of b . Thus, increasing the value of b increases the number of

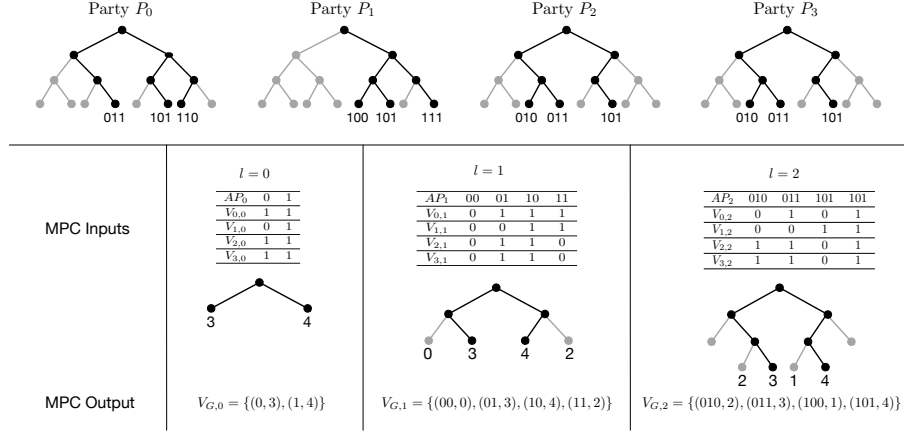


Fig. 4: **Example execution.** For parameters $n = 4$ parties, $\kappa = 3$, $|L_i| = 3$, $b = 2$, and $\tau = 3$. For ease of exposition, we assume all parties have coin values set to 1. At the top, we show the inputs of each party P_i — the black paths denote the paths to input transactions, and the remaining paths are greyed out. In the first iteration, there are only two active prefixes $AP_0 = \{0, 1\}$. Parties input a 1 to a path they are interested in exploring. All parties are interested in exploring both paths except party P_1 , which does not want to explore path 0 since none of its inputs start with that prefix. At the end of the computation, parties output $V_{G,0}$ with 3 parties voting to explore prefix 0 and 4 parties voting to explore prefix 1. Since both these values exceed threshold τ , all four paths are explored in the next iteration $l = 1$. Similarly, at the end of this iteration, only two paths 01 and 10 stay active. Finally, at the end of the execution, the parties output 011 and 101 as the output transactions.

prefixes processed per level, but the number of levels reduces to $\log_b 2^\kappa$. Since κ affects only the correctness of the protocol and not its security, we can set κ smaller to speed up the protocol, balancing the probability of collisions where it is unclear which of two or more potential transactions are intended to be on the list. These collisions are not fatal to the protocol because if an unnecessary transaction is added, the correct one can be added in the next iteration of the inclusion list protocol.

Figure 4 illustrates an execution of the semihonest component of our protocol. See Appendix B for more optimizations of this protocol idea.

3.2 Protecting Against Malicious Behavior

As described thus far, our protocol is vulnerable to three broad classes of potential attacks. An attacker can do any of the following:

1. manipulate its inputs at each level of the protocol.
2. incorrectly sum the values received from other parties.
3. stop responding to messages or send inconsistent values to different parties.

To make the first class of attacks concrete, an attacker could artificially increase the count for a particular node in the tree in order to include an extra

transaction in the final list, or even worse, an attacker could send values that “cancel out” the counts sent by other committee members, thereby preventing transactions from making it onto the final inclusion list. We avoid these situations by augmenting the MPC used at each level of the tree. In addition to computing and revealing sums, we check that each indicator bit value is in fact a sharing of one or zero. For a bit b , this is just a matter of computing $b \cdot (1 - b)$ in the MPC and revealing the result, which should be 0.

Next, we need to ensure that committee members follow the protocol honestly. This can be achieved using a maliciously-secure MPC protocol with abort (solving problem 2 above) or a robust MPC (solving problems 2 and 3), as described in Section 2.3.

Running the robust and non-robust MPC protocols in parallel means that a block can either immediately have its inclusions list (in the optimistic case) or that an inclusion list is guaranteed to become available for a future block (pessimistic case). A detailed description of this mechanism is provided in Appendix A.3, and a security analysis of the protocol is presented in Appendix A.4.

4 Evaluation

In this section, we evaluate the performance of our inclusion list protocol described in Section 3 and formalized in Figure 2.

Implementation details. We implemented the protocol in two multiparty computation frameworks: MP-SPDZ [18] (referred to as **MP-SPDZ**) and HoneyBadgerMPC [20] (referred to as **HB-MPC**). MP-SPDZ is a framework that supports multiple protocols; we used the Mascot protocol [19], using which we obtain optimistic correctness, and also soundness, anonymity and plausible deniability even under $t < n/3$ malicious parties. HoneyBadgerMPC, in addition, supports the correctness property. We present additional implementation details in Appendix B.1. Our code is available at <https://anonymous.4open.science/r/MP-SPDZ-C9DF/> for MP-SPDZ and https://anonymous.4open.science/r/Partial_HoneyBadgerMPC/ for HB-MPC.

Experimental setup. We conducted our experiments on four machines connected on a LAN. Each machine is equipped with an Intel Xeon Gold 6248R CPU @ 3.00GHz, running on VMware virtualization. Each of the machines have 16 virtual CPUs, 10Gbps bandwidth, and 256 GB of disk storage. One of the machines has 64 GB RAM whereas the rest have 48 GB RAM.

We note that based on the functionality provided by the underlying framework, the experiments on MP-SPDZ utilized all four machines equally, simulating multiple parties per machine when needed, whereas the ones on HoneyBadgerMPC were run on a single machine. Moreover, for MP-SPDZ, each party’s instance could utilize multiple CPU cores, when available, to improve performance.

Baselines. We consider two different baseline algorithms to compare our solution. We implement both baselines using MP-SPDZ. The first baseline, **base-naïve**, is a naïve baseline where, for every input transaction from every party, we compare

it with every other transaction to check if it appears at least τ number of times. We do not perform a shuffling step, which would be required to hide which user included which transactions, and thus this baseline reports a lower bound on the performance of a real baseline. The protocol is described in more detail in Appendix A.1. The second baseline, **base-mpcc**, follows the protocol in Figure 2 but performs the threshold comparison at line 23 within the MPC instead. This hides the frequency of prefixes but incurs a larger number of multiplication gates to perform the comparison.

Experiment parameters and metrics. We choose our parameters in line with EIP-7805 [22], which considers the proposed implementation for inclusion list design (without privacy) for Ethereum. Thus, we consider $n = 16$. We set the mempool size to 128 and let the parties choose transactions from this mempool s.t. $|L_i| = 100$ for all parties P_i . We set $\kappa = 64$; thus, we consider the first 64 bits of a transaction hash in our tree exploration. Moreover, we choose $b = 4$ empirically as the default parameter since it yields better performance (as will be shown later).

The primary metric of interest is the latency for running the inclusion list protocol; thus we report the maximum time across all honest parties for the online portion of the MPC protocol. For each parameter setting, for **MP-SPDZ** and **base-mpcc**, we repeated the experiments five times and report the mean latency (across all parties and all runs) together with 95% confidence intervals. For **base-naïve** and **HB-MPC**, since they have a longer running time, we executed these experiments only once; we present the mean latency across all parties.

4.1 Evaluation Results

We assess three primary questions through our evaluation.

- Q1.** How efficient is the inclusion list protocol in the default setting?
- Q2.** How does the protocol performance change with change in parameters such as n , κ , τ , and $|L_i|$?
- Q3.** How does the protocol’s performance compare to the baselines **base-naïve** and **base-mpcc**?

Efficiency in the default setting. In the default setting, the **MP-SPDZ** implementation runs in 5.1 ± 0.1 s whereas the **HoneyBadgerMPC** implementation runs in 432.7s. Thus, the execution time for achieving optimistic correctness is within the 12s slot time for Ethereum. The computation time of the more efficient **MP-SPDZ** implementation is dominated by 16,384 MPC multiplications distributed over the 32 sequential levels of execution. Note that running our protocol in a WAN setting will incur additional network delays, but this cost can be compensated for via speculative execution and short trees, two optimizations we describe in Appendix B.2.

Protocol performance as a function of parameters. Figures 5 and 6 present the performance of **MP-SPDZ** and **HB-MPC** as a function of n , κ , τ , and $|L_i|$.

First, we observe that as n increases, the execution time for both protocols increases. For MP-SPDZ, the amount of data transferred and computed upon increases roughly linearly with increase in n . For HB-MPC, the communication increases linearly and the computation increases super-linearly in n .

Second, as κ increases, we observe that the execution time increases roughly linearly. Every level of the tree, except for the first few levels, perform roughly similar amount of computation corresponding to the max number of prefixes/-transactions processed per level. In the first few levels, the number of prefixes processed per level increases until it reaches that threshold. Thus, we observe a roughly linear change in latency with linear change in κ .

Third, for both protocols, $b = 4$ performs better than $b = 2$ and $b = 16$. Observe that a larger b implies checking for a larger number of prefixes at once; thus, this increases the amount of communication and computation per level. At the same time, for a fixed κ , the height of the tree is $\log_b \kappa$, and thus, the number of levels decreases.

Finally, the number of transactions $|L_i|$ in each party's input does not affect performance very much in our implementation. For MP-SPDZ, this is due to a framework constraint where the input size processed at each level must be hard-coded in advance, so we fix the size at a reasonable upper bound. See Appendix B.1 for details. For HB-MPC, the cost of opening shares in batches appears to dominate the computation, rendering changes in input size less important.

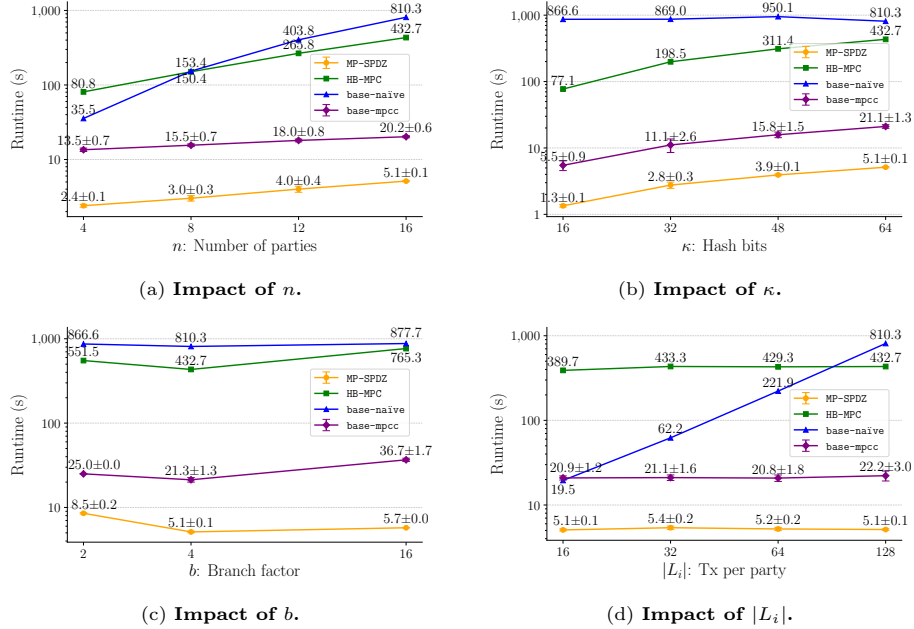


Fig. 5: Performance as a function of parameters. Figure 6 compares MP-SPDZ and base-mpcc on a linear scale.

Comparison with baselines. Figures 5 and 6 compare the performance of MP-SPDZ with baselines **base-naïve** and **base-mpcc**. First, we execute the **base-naïve** protocol only for a subset of data points since its performance is much worse than MP-SPDZ. In particular, under the default parameterization, it is 160x factor worse than MP-SPDZ. Second, MP-SPDZ performs 3x-7x better than **base-mpcc** under all parameterizations. This is due to avoiding the additional computation and communication incurred due to performing a threshold comparison under MPC for each prefix on a level of the tree.

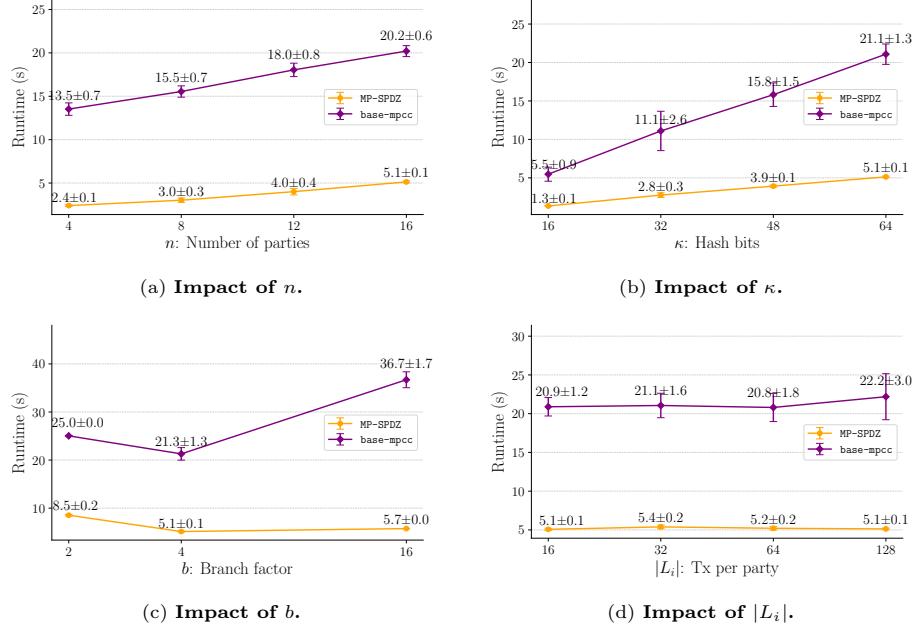


Fig. 6: Performance as a function of parameters between MP-SPDZ and **base-mpcc**. Presents the same data as Figure 5 for MP-SPDZ and **base-mpcc** on a linear scale.

5 Conclusion

We have presented a new protocol to achieve privacy-preserving inclusions lists. Our protocol allows members of an inclusion list committee to collectively construct an inclusion list without revealing which transactions they have chosen to include and while maintaining plausible deniability that they did not contribute any particular transaction. In the presence of centralization of block proposing power, our hope is that our protocol can allow committee members to prevent retaliation and ensure robust censorship resistance.

References

1. Credit and debit card market share by network and issuer. <https://www.fool.com/money/research/credit-debit-card-market-share-network-issuer/>, 2025. Accessed: 2025-08-21.
2. 0xF1D. Anonymous inclusion lists (anon ils). <https://ethresear.ch/t/anonymous-inclusion-lists-anon-ils/19627>, 2024. Accessed: 2025-08-21.
3. 0xF1E. zkfocil: Inclusion list privacy using linkable ring signatures. <https://ethresear.ch/t/zkfocil-inclusion-list-privacy-using-linkable-ring-signatures/21688>, 2025. Accessed: 2025-08-21.
4. Donald Beaver. Efficient multiparty protocols using circuit randomization. In *CRYPTO*, 1991.
5. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *CCS*, 1993.
6. Ward Beullens, Shuichi Katsumata, and Federico Pintore. Calamari and falafl: logarithmic (linkable) ring signatures from isogenies and lattices. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 464–492. Springer, 2020.
7. Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Lightweight techniques for private heavy hitters. In *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*, pages 762–776. IEEE, 2021.
8. David Chaum. The dining cryptographers problem: Unconditional sender and recipient untraceability. *J. Cryptology*, 1(1):65–75, 1988.
9. Henry Corrigan-Gibbs and Bryan Ford. Dissent: accountable anonymous group messaging. In *Proceedings of the 17th ACM Conference on Computer and Communications Security, CCS 2010, Chicago, Illinois, USA, October 4-8, 2010*, pages 340–350. ACM, 2010.
10. Henry Corrigan-Gibbs, David Isaac Wolinsky, and Bryan Ford. Proactively accountable anonymous messaging in verdict. In *Proceedings of the 22th USENIX Security Symposium, Washington, DC, USA, August 14-16, 2013*, pages 147–162. USENIX Association, 2013.
11. Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, volume 7417 of *Lecture Notes in Computer Science*, pages 643–662. Springer, 2012.
12. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO*, 1986.
13. Elijah Fox, Malleh M. Pai, and Max Resnick. Censorship Resistance in On-Chain Auctions. In Joseph Bonneau and S. Matthew Weinberg, editors, *5th Conference on Advances in Financial Technologies (AFT 2023)*, volume 282 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 19:1–19:20, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
14. Michael J. Freedman, Kobbi Nissim, and Benny Pinkas. Efficient private matching and set intersection. In *EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 1–19. Springer, 2004.
15. Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In *STOC*, 1987.

16. Philippe Golle and Ari Juels. Dining cryptographers revisited. In *Advances in Cryptology - EUROCRYPT 2004, International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004, Proceedings*, volume 3027 of *Lecture Notes in Computer Science*, pages 456–473. Springer, 2004.
17. Jens Groth and Markulf Kohlweiss. One-out-of-many proofs: Or how to leak a secret and spend a coin. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 253–280. Springer, 2015.
18. Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 1575–1590. ACM, 2020.
19. Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: faster malicious arithmetic secure computation with oblivious transfer. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 830–842. ACM, 2016.
20. Donghang Lu, Thomas Yurek, Samarth Kulshreshtha, Rahul Govind, Aniket Kate, and Andrew Miller. Honeybadgermpc and asynchromix: Practical asynchronous MPC and its application to anonymous communication. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, pages 887–903. ACM, 2019.
21. Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, 1979.
22. Thomas Thiery, Francesco D’Amato, Julian Ma, Barnabé Monnot, Terence Tsao, Jacob Kaufmann, and Jihoon Song. Eip-7805: Fork-choice enforced inclusion lists (focil). Ethereum Improvement Proposal (EIP), 2024. Draft (Standards Track, Core), created 1 November 2024.
23. U.S. Department of the Treasury. OFAC Specially Designated Nationals Data, 2022.
24. Jelle Vos, Mauro Conti, and Zekeriya Erkin. Sok: Collusion-resistant multi-party private set intersections in the semi-honest model. In *IEEE Symposium on Security and Privacy, SP 2024*, pages 465–483. IEEE, 2024.
25. Anton Wahrstätter, Jens Ernstberger, Aviv Yaish, Liyi Zhou, Kaihua Qin, Taro Tsuchiya, Sebastian Steinhorst, Davor Svetinovic, Nicolas Christin, Mikolaj Barczeniewicz, and Arthur Gervais. Blockchain Censorship, June 2023. arXiv:2305.18545 [cs].
26. Fredrik Winzer, Benjamin Herd, and Sebastian Faust. Temporary censorship attacks in the presence of rational miners. In *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 357–366. IEEE, 2019.
27. Fei Wu, Thomas Thiery, Stefanos Leonardos, and Carmine Ventre. From competition to centralization: The oligopoly in ethereum block building auctions, 2024.
28. Sen Yang, Kartik Nayak, and Fan Zhang. Decentralization of ethereum’s builder market, 2025.
29. Burak Öz, Danning Sui, Thomas Thiery, and Florian Matthes. Who wins ethereum block building auctions and why?, 2024.

A Additional Protocol Details

A.1 A Naïve Solution

A naïve approach to solving this problem is to perform a threshold private set intersection using a multiparty computation protocol (described in Figure 7). In particular, for every transaction held by every party, we check if the transaction appears in at least $\tau - 1$ other inputs. If so, we output the transaction. Performing such a computation, though, will require $O(n^2|L_i|^2)$ comparisons where each comparison requires at least $O(\kappa)$ multiplication gates (where κ is a security parameter corresponding to the length of transaction hashes) under MPC even if we ignore the cost of checking for the threshold to reach τ . Since multiplications of secret shared data are the most expensive operation in an MPC, we seek to minimize the number of multiplications, even at the cost of more additions, which are effectively free.

```

1: out := {}
2: for  $i := 1 \dots n$  do
3:   for each entry  $e \in L_i$  do
4:     cnt = 0
5:     for each entry  $e' \in \cup_k L_k$  do
6:       if  $e = e'$  then
7:         cnt := cnt + 1
8:       if cnt  $\geq \tau$  then
9:         out := out  $\cup \{e\}$ 
10: Return out

```

Fig. 7: $\mathcal{F}_{\text{naïve}}$

An advantage to this approach is that it does not leak any additional information about the parties' inputs. However, it does incur a large number of comparisons and, thus, multiplication gates. Note that our goal is to only hide the mapping between transactions and the parties that input it. Thus, in comparison to this naïve solution, our solution computes a union of all inputs and reveals the frequency with which each input appears. From a performance standpoint, this will not require performing comparisons similar to the naïve approach under MPC. Thus, such an approach would reduce the computation primarily to additions, which are less expensive than multiplications. We can then rely on a post-processing step in the clear that filters transactions that appear at least τ times. From a privacy standpoint though, if a given input appears too many times, e.g., n times, then no honest party can deny that they did input the value. We address this deniability issue separately, as discussed in the following subsection.

In our evaluation, we compare the results of our approach (referred to as MP-SPDZ and HB-MPC) with an oblivious implementation of the naïve approach from Figure 7 (referred to as **base-naïve**). Observe that implementing line 9 of

base-naïve under MPC (to preserve anonymity and deniability) requires either using an oblivious list or naively adding every element to the output and securely shuffling the output. Our implementation does not perform the shuffling step and thus, the reported performance is a lower bound on the actual performance incurred by this algorithm.

A.2 Ensuring Deniability

In our protocol, at each level, $\mathcal{F}_{\text{MPC}}$ reveals the frequency with which a given prefix appears (line 20), allowing us to perform a cleartext comparison with threshold τ (line 23). However, if this frequency is large, e.g., n , then all parties lose deniability because an observer can discern that everyone on the committee included a transaction with that prefix. From a privacy standpoint, this can be easily addressed by performing the required comparison with τ within $\mathcal{F}_{\text{MPC}}$; however, computing a comparison inside an MPC incurs additional multiplication gates, increasing the communication, round, and time complexity of each level of the protocol.

We devise a lightweight mechanism to address this issue. Observe that the issue occurs if the revealed frequency is $\geq n - t$ since $\leq t$ of the inputs are adversarial and the adversary may input 0; thus, if we can guarantee that the output is always $\leq n - (t + 1)$, then an honest party has plausible deniability. We achieve this by ensuring that at least $t + 1$ parties, or at least 1 honest party, are secretly designated to input 0 to explore any path for all levels; we will call them *zero-input parties*. This guarantees that the frequency for any prefix never exceeds $n - 1$ (note that adversarial zero-input parties can still choose to send their inputs; if they indeed input 0s, then the prefix never exceeds $n - (t + 1)$). Since the identity of the zero-input parties remains secret, any party can claim to have been one of them, maintaining plausible deniability.

The remaining concern is determining the zero-input parties. Our solution requires every party to sample a λ 0/1 coins uniformly at random (Figure 2 line 8): a 0 implies that the party is a zero-input party, whereas a 1 implies that the party will send its input as is. We then sum up each of the λ coin values across the n parties using MPC and reveal the output out_{coin} (line 9). Eventually, our goal is to ensure that the number of zero-input parties are $\geq t + 1$ and $\leq x$ for some parameter x . Thus, we choose the first coin at the lowest index j among the λ coins s.t. $n - x \leq \text{out}_{\text{coin}}[j] \leq n - (t + 1)$. Observe that the inequality $n - x \leq \text{out}_{\text{coin}}[j]$ ensures that there are not too many zero-input parties, which would prevent honest parties from being able to include transactions on the inclusion list. Thus, the parameter x provides a trade-off between plausible deniability and the number of honest parties that must input a transaction for it to be guaranteed in the output.

A.3 Achieving Malicious Security

In this section, we discuss the protocol in the malicious setting where the adversarial parties may deviate arbitrarily from the prescribed protocol.

Recall that our goal is to obtain correctness, soundness, and privacy for the honest parties in the presence of such an adversary. We now discuss several steps taken towards achieving these properties.

Using MPC secure under malicious adversaries. First, we ensure that the underlying $\mathcal{F}_{\text{MPC}}$ functionality described in Figure 3 is implemented under an MPC protocol tolerating t *malicious* adversaries. In our eventual implementation, we consider two implementations Mascot protocol [19] as implemented in the mp-spdz library [18] and HoneyBadgerMPC [20].

Addressing adversarial inputs. Second, observe that in an MPC protocol, the adversary has input independence, i.e., it can input anything to the protocol. By doing so, the adversary can potentially attempt to disrupt the correctness or privacy of the computation. There are two key concerns:

1. **Incorrectness through malformed inputs.** A malicious party may input arbitrary values that interfere with the correct aggregation of votes, potentially causing transactions that should have been considered to be excluded. For instance, if the adversary, instead of sharing a 0/1 value, if it shares a different value, e.g., -2, this can affect the correctness as well as the privacy of the protocol.
2. **Inconsistent inputs.** The adversary can send inconsistent values across different levels of the protocol. As an example, consider the case where a party votes 1 at level l for a given active prefix $AP_l[p]$, thereby claiming to have transactions matching that prefix. The same party may then later vote 0 at all children vertices of this prefix, which is inconsistent with its earlier input and violates the intended protocol semantics. This can potentially cause additional paths to be explored due to “fake” inputs while honest execution prunes branches that do not meet the voting threshold or vice versa. A malicious party that repeatedly votes “1” for all active prefixes can force the exploration of many unnecessary paths.

For the first case, we enforce input validity through a lightweight bit-checking mechanism. Each party is restricted to submitting only binary values $\{0, 1\}$ to indicate its interest in extending a prefix. If a party attempts to inject a non-binary value, the check detects the deviation and causes the computation to abort. Concretely, for each prefix p in party i ’s input $V_{i,l}$ at level l , we ensure $V_{i,l}[p] \cdot (1 - V_{i,l}[p]) = 0$ (Figure 3 (left), lines 1-4). We emphasize that this check is performed over secret-shared values, ensuring that no information about honest inputs is leaked. A similar check is also performed to check for coin values when determining zero-input parties (Figure 3 (right) lines 1-4).

For the second case, note that this behavior does not compromise security guarantees. Since at most t parties are malicious, each can contribute at most one vote per transaction. Thus, a spurious transaction proposed only by malicious parties cannot reach the inclusion threshold, and will never appear in the final list. The effect is limited to performance: In the worst case, the number of paths explored is bounded by $O(|L|) \times b$, which is bounded by the scenario in which all

transactions in the mempool are explored. Given that typical mempool sizes are on the order of 100–200 transactions, this overhead is not prohibitive.

Ensuring correctness. With the above changes, the protocol achieves optimistic correctness and privacy. The remaining consideration is correctness under arbitrary behavior by the adversary. MP-SPDZ [18] and several other MPC protocols only provide malicious security with abort — this means that if some malicious party chooses not to participate in a protocol step, then the protocol stalls. In practice, we need to guarantee robustness, i.e., we want guaranteed output delivery even when t parties behave arbitrarily. To address this requirement, we rely on a recent work, HoneyBadgerMPC [20], which is Shamir Secret Sharing-based MPC protocol [21].

A.4 Security Analysis

This section revisits our the security and functionality properties required of our scheme and shows why our scheme satisfies each one.

Theorem 1. *Assuming it is instantiated with an MPC that satisfies malicious security with abort and parameters are set such that $n - x > \tau + t$, $x > t + 1$, and $y \geq \tau + x$, our scheme satisfies optimistic correctness with all but negligible probability in the choice of parameter λ .*

Theorem 2. *Assuming it is instantiated with an MPC that provides robustness and parameters are set such that $n - x > \tau + t$, $x > t + 1$, and $y \geq \tau + x$, our scheme satisfies correctness with all but negligible probability in the choice of parameter λ .*

We distinguish between correctness and optimistic correctness in our scheme. Both properties require that a value v included in least y honest parties’ input lists L_i must appear in the output list L . The distinction lies in expectations of malicious behavior. Optimistic correctness requires this property to hold in the case that all parties follow the protocol specification honestly, and (non-optimistic) correctness expects this property to hold in the presence of t Byzantine parties.

The optimistic correctness of our scheme follows by construction. We will assume first that there are enough honest parties that are not zero-input parties so that more than τ honest parties are giving inputs based on their actual lists L_i . At each level of the tree of possible list items explored by the protocol, the parties will continue to explore the path leading to any transaction hash that has more than τ votes. By induction over the levels of the tree, we find that any transaction with more than τ votes will be included in the final list.

However, while the core protocol has perfect optimistic correctness, the introduction of zero-input parties introduces some chance of failure. In particular, if all λ executions of the coin tossing process result in a sum $\text{out}_{\text{coin}}[j]$ that is not between $n - x$ and $n - (t + 1)$, then the protocol will fail. We also need that $n - x > t + \tau$, or else there may be so many zero-input parties that it will not be possible for honest parties to include any transactions. Thus for the interval of

allowed values of $\text{out}_{\text{coin}}[j]$ to be non-empty, we need that $n - (t + 1) - (n - x) > 0$, which implies that $x > t + 1$. Since we assumed that parameters are set such that this condition is met, there is a constant probability of having an acceptable value of $\text{out}_{\text{coin}}[j]$ for each coin toss, and repetition reduces the failure probability to negligible in λ . A larger value of x increases the probability of a successful coin toss sum in each instance of the protocol, reducing the necessary size of λ and therefore slightly reducing communication costs. From the bounds above, we have that $t + 1 < x < n - t - \tau$.

Finally, since the correctness definition says that y honest parties have a value v in their input, we additionally need to ensure that there are enough honest parties remaining to include transaction v after the coin tossing protocol. This means that even if all zero-input parties whose inputs are excluded are honest parties who had the value v in their inputs, there must remain τ such parties. Thus we need $y \geq \tau + x$.

Correctness of the construction in the presence of Byzantine parties arises from the use of a slower but robust MPC in parallel with the MPC offering malicious security with abort. Since the robust MPC is resilient to attacks from byzantine parties, the output of the MPCs will still be delivered to the parties and the protocol can proceed despite malicious behavior.

Note that we have not established a fixed relationship between t and τ for the purposes of our correctness theorems, but we generally expect that $\tau > t$. This is required for other security properties. This has ramifications for the relationship between n and t as well. In general, lower bounds on robust MPC require that $t < n/3$. Given these additional constraints, the inequalities in our correctness theorem require that $x = n/3$ as well, which results in a requirement that the number of non-zero input parties be exactly $2n/3$. This is fine for small n , but as n grows, the probability of this event becomes negligible. Thus we would need $t < n/(3 + \epsilon)$ in this case, for a constant ϵ , in order for a reasonably-sized λ to give a high degree of correctness.

Theorem 3. *Assuming the malicious security (with abort) of the underlying MPC protocol, and that $\tau > t$, our inclusion list protocol satisfies soundness.*

Our protocol achieves soundness as long as $\tau > t$, i.e., when there are not enough malicious parties to include arbitrary transactions on the input list on their own. Observe that the malicious security with abort (or robustness) of the MPC scheme used to instantiate our inclusion list protocol ensures that any outputs received from the protocol will be the result of the correct computation of the underlying MPC functionality. Our MPC functionality includes a bit check process which requires that each input is always either a 0 or 1. The combination of these two properties means that, at all times, a malicious party who participates fully in the protocol cannot affect the output beyond the setting of its own inputs. But since $\tau > t$, there is no setting of malicious inputs that can cause a transaction hash to be included without any honest parties also including that hash.

Theorem 4. *Assuming the malicious security (with abort) of the underlying MPC protocol, our inclusion list protocol satisfies anonymity.*

The anonymity of our scheme arises almost directly from the security of the underlying MPC protocols. Observe that the view of each party consists only of the messages in each invocation of an MPC. Except for the outputs of the MPCs, these messages can all be simulated without knowledge of any inputs. Thus the MPC outputs are the only messages of concern. But these are all either the results of bit checks – which are either zeros or a function of only a misbehaving party’s input – or the sums of parties’ inputs in each layer of the MPC. But the count of how many parties included a transaction is permissible leakage in our anonymity definition, so this does not pose a problem.

Theorem 5. *Assuming the malicious security (with abort) of the underlying MPC protocol and that $n - (t + 1) > 0$, our inclusion list protocol satisfies plausible deniability.*

Plausible deniability follows directly from our coin flipping protocol. Since we require that there be some $\text{out}_{\text{coin}}[j] < n - (t + 1)$ (which is possible because $n - (t + 1) > 0$), this means that there must always be at least one party that is designated as a zero-input party. Since the MPC hides who gives which inputs to the protocol, any honest party can claim to have been the zero-input party. If a party is a zero-input party, then their inputs to the protocol are independent of their actual input list L_i , so they can claim any possible list as their input. In particular, they can claim that their list did not include any sensitive transaction they wish to deny.

B Implementation and Optimizations

We implemented our malicious-secure privacy-preserving inclusion list protocol using the MASCOT protocol [19] as implemented by `mp-spdz` [18], and we use `HoneyBadgerMPC` [20] as the robust backup for the variant of the protocol that requires one. This section describe performance-related design decisions and optimizations that improve performance over a straightforward implementation of the scheme as described in the preceding sections. We present several potential optimizations and indicate which ones we included in our implementation.

B.1 Implementation Details

This section describes practical implementation decisions that do not affect the overall description of the protocol but result in improved performance.

Interacting with `mp-spdz`. The `mp-spdz` library incurs some overhead to setup a new protocol each time an MPC starts. This means that running a separate MPC protocol per level of our protocol would result in repeatedly incurring the setup cost. Instead, we run a single MPC protocol that takes additional inputs and returns outputs at each level. However, `mp-spdz` only allows inputs to MPC protocols from static input files or command-line input. Thus we use the python `pexpect` module to send and receive inputs to the MPC protocol on a simulated command line rather than using file I/O for inputs.

This implementation decision comes with a tradeoff, as the library needs to know how many inputs to take at compile time (python-like MPC code is compiled into a final protocol). This means the protocol will need to take in a number of inputs corresponding to the maximum expected inclusion list size at each level, discarding inputs that end up not being needed. We find that this cost is preferable to that of paying the protocol initialization cost at each level.

Adding concurrency on top of HoneyBadgerMPC. We find that HoneyBadgerMPC runs in a single-threaded mode by default, relying on batching operations rather than concurrency in order to improve performance. While we ran our evaluation in a setting with only one core available per party, we briefly describe how to modify our implementation in order to take advantage of parallelism without altering the protocol. To do this, we could implement our protocol to run in separate MPC processes, each running on a different core. Consider the case of exploring a tree with branching factor four. We can think of each of the four nodes in the first level of the protocol as themselves being the root of a smaller instance of the same protocol, and run each one as a separate process. Since the outcomes of separate paths down a tree are independent of each other, this has no impact on the computation or results of the protocol, except to allow different parts of the protocol to run in parallel on different cores. The same idea applies for any branching factor, so we can split the protocol into as many subproblems as we need to optimally use the computing resources available. Moreover, the branching factor at this first level need not match the branching factor used within each instance of the protocol, allowing for even greater flexibility.

B.2 Protocol Optimizations

This section describes protocol modifications that trade off some security in exchange for improved performance. We evaluated our protocol without these optimizations in order to show performance with the highest security level, but we present them here as they may provide favorable tradeoffs for deployments of this protocol.

Speculative execution of subprotocols. We can reduce the overall number of protocol rounds considerably via a speculative subprotocol execution optimization. Since checking that all the sharings consist of 0/1 values requires a round of communication to compute the necessary multiplication, each subprotocol now requires an additional communication round to complete this step. However, the output sums of each subprotocol can be computed before these checks are completed. As such, the next subprotocol can proceed assuming that the checks will work out. In the case that they do, the overall rounds of the protocol are reduced, and in the case that they do not, the additional speculative round will not reveal anything about honest users’ inputs because of the security properties of the MPC. Thus our protocol will at all times have a pipeline of subprotocols in progress.

This optimization comes with a mild security tradeoff. Since subprotocols execute speculatively without waiting for all the bit checks of a previous level to

complete, it is possible for a malicious party to temporarily inflate/deflate counts for a node in the tree by sending inputs that are not zero or one. While this will eventually be caught, in the meantime, the protocol may explore parts of the tree that do not have sufficient votes to be included, thereby revealing additional inclusion counts that would not have been visible in the honest protocol (while still not revealing which parties are responsible for those counts). This may or may not be a desirable tradeoff for a given deployment scenario.

Reducing rounds with short trees. We can reduce the round complexity of our protocol by breaking the output of the hash function H into two halves of length $\kappa/2$ and running the protocol separately on each half in parallel. Then a transaction is only included in the inclusion list if both halves of its hash are present in the protocol output. This increases collision probability but cuts the number of rounds needed in half because the number of protocol rounds depends directly on the length of the hash, which determines the number of protocol levels.

Concretely, the probability of a collision in a list of q transaction is upper bounded by $q^2/2^\kappa$ for a κ bit hash. Since collisions in the left and right halves of a hash function modeled as a random oracle [12,5] occur independently, collisions will occur with probability at most $\frac{q^2}{2^{\kappa/2}} \cdot \frac{q^2}{2^{\kappa/2}} = \frac{q^4}{2^\kappa}$.

To make the reduction in collision probability less severe, we instead split the output of H into three overlapping bit strings of length $\kappa/2$, requiring a transaction be included in all three halves in order to be included in the list. That is, we run the protocol with the first and second halves of the output of H , but also with the bits in positions $\kappa/4$ to $3\kappa/4$. This increases the computation per round but allows using a smaller value of κ to get the same concrete security level.

C Comparison of Potential Approaches

This section compares our approach qualitatively to other proposed or potential approaches to private inclusion lists. Given that there are not concrete proposals or implementations to which we can compare, an empirical comparison is unfortunately impossible. Nonetheless, we hope this discussion helps to frame the tradeoffs results in the context of other related approaches.

Ethereum proposals. Section 1 briefly discussed approaches that have been proposed for the Ethereum blockchain [2,3], so we do not repeat that discussion here, except to observe that these proposals rely on an ideal anonymous broadcast protocol to protect anonymity at the network layer. One proposal [3] leverages the ideal anonymous broadcast to target a stronger security goal than is pursued here, namely a setting where the identities of inclusion list committee members are hidden among an even larger anonymity set of non-members. It is an interesting direction for future work to pursue this privacy goal without relying on heavyweight cryptographic primitives or an ideal anonymous broadcast protocol.

Inclusion lists from other primitives. Since our protocol provides an efficient way for committee members to anonymously send each other the contents of their inclusion lists, one alternative would be to rely on a classic approach like DC-nets [8], whereby committee members could anonymously broadcast the contents of their inclusion lists and then count up the number of times each transaction has appeared. In a semihonest setting, this approach would require more communication than our scheme, but would otherwise be feasible. However, when considering malicious clients, it becomes more challenging. Malicious clients may try to send random messages to clobber others' contributions or send the same transaction(s) multiple times to inflate their count. While techniques exist to ensure that participants in a DC-net cannot corrupt the outputs of the protocol [16,9,10], additional ideas would be needed to ensure that parties cannot enter duplicate inputs.

Another approach to inclusion list construction would be to rely on protocols for private set intersection [14]. While two-party PSI protocols have become increasingly efficient, multiparty protocols lag behind in performance [24]. One of our naïve baselines was to build a PSI-type protocol on top of MPC, and perhaps this approach could be refined to build more efficient protocols. Specifically, inclusion list formation as described here can be thought of as a multi-party PSI cardinality problem. In fact, there is such a protocol implicit in our construction. Just as our core semihonest protocol needs additional protections for malicious behavior and to preserve deniability, any PSI protocol would need to be modified to provide similar features as well.